

OOT_设计模式

1. Visitor 访问者模式：编译器 AST 分析

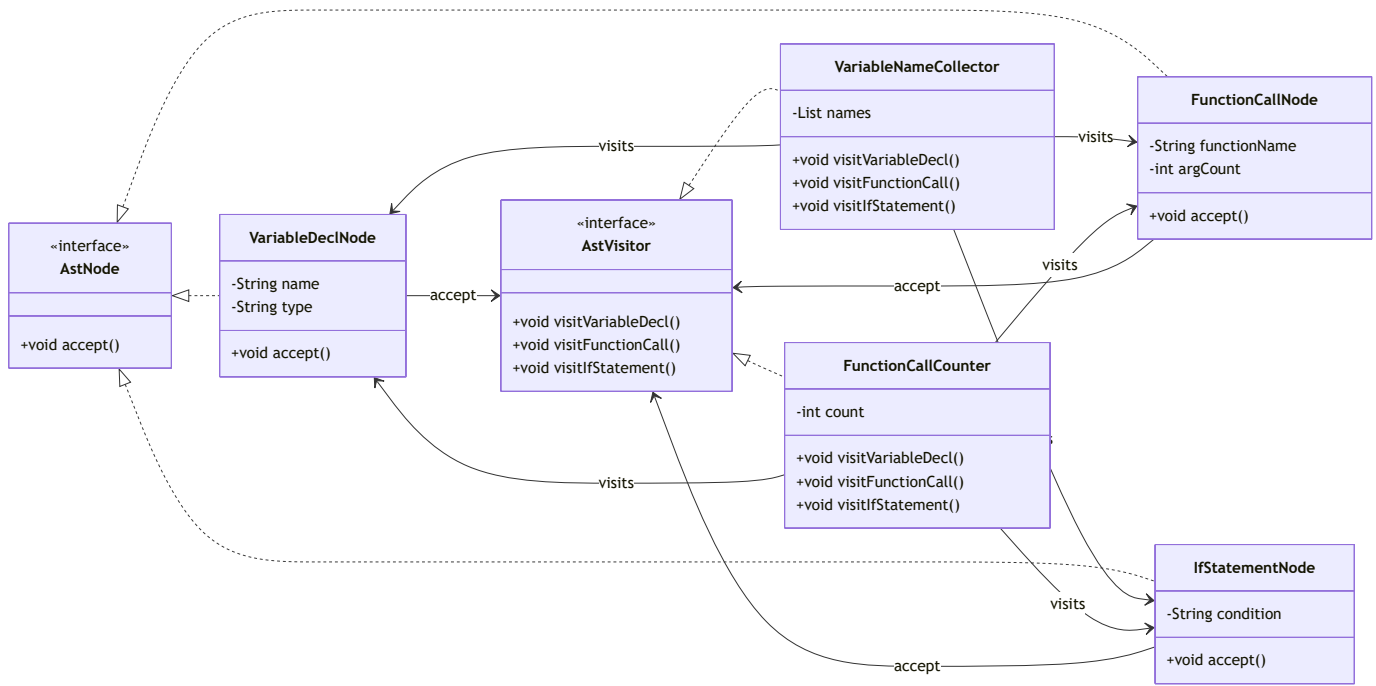
小题背景

一个编译器中有多种 AST 节点，例如变量声明节点、函数调用节点、条件语句节点。现在需要在不修改这些节点类的情况下，增加不同分析功能，例如收集变量名、统计函数调用次数、检查条件语句复杂度。

应使用的模式

Visitor 访问者模式

Mermaid UML



设计说明

AST 节点结构相对稳定，但分析操作可能不断增加。让每个节点实现 `accept()`，把具体分析逻辑放到访问者类中。新增分析功能时，只需要增加新的 `AstVisitor` 实现类，不需要修改原有节点类。适合“对象结构稳定，操作频繁变化”的场景。

2. Observer 观察者模式：股票价格推送

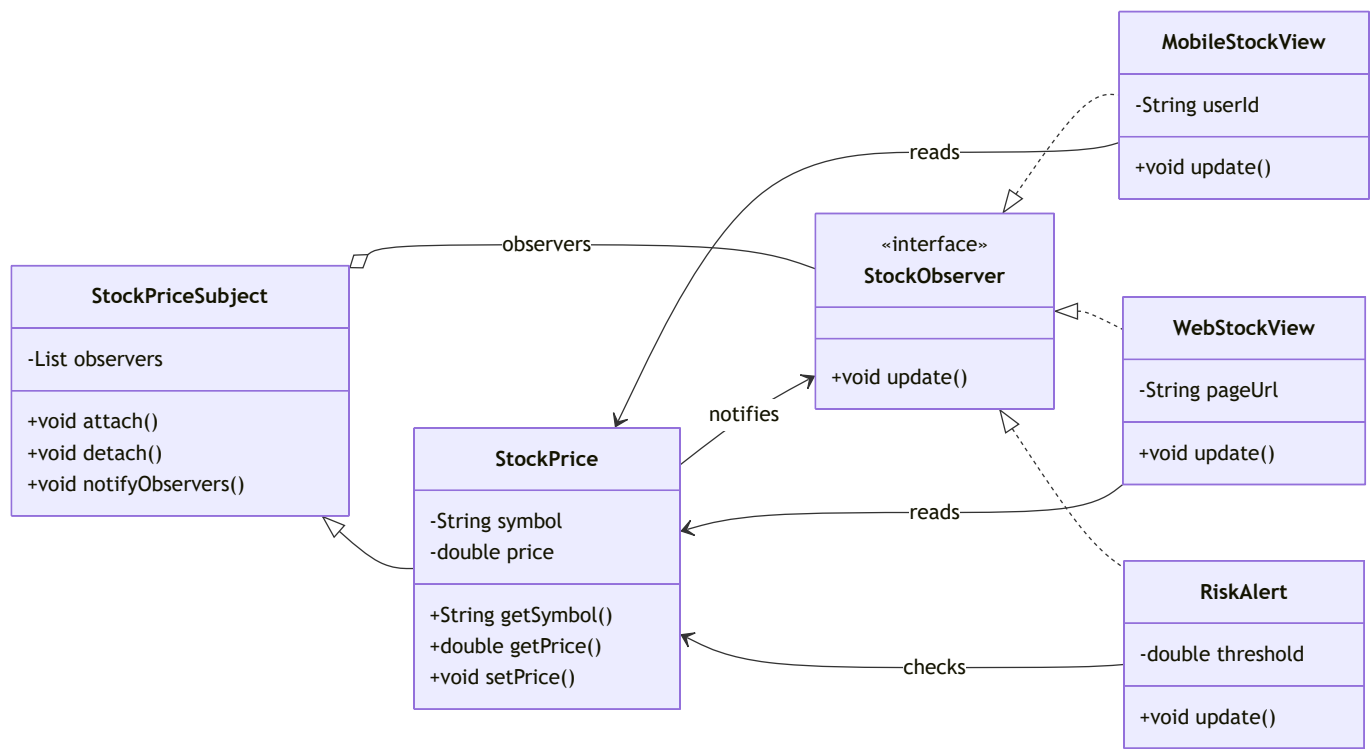
小题背景

一个股票行情系统中，`StockPrice` 的价格变化后，需要自动通知多个界面组件，例如手机 App 显示器、网页显示器、风险预警器。

应使用的模式

Observer 观察者模式

Mermaid UML



设计说明

股票价格是被观察者，多个界面或预警模块是观察者。`StockPrice` 变化时调用 `notifyObservers()`，所有观察者通过 `update()` 接收通知。这样股票类不需要知道具体有哪些显示器或预警器，降低了耦合，也方便动态添加或删除观察者。

3. Strategy 策略模式：订单折扣计算

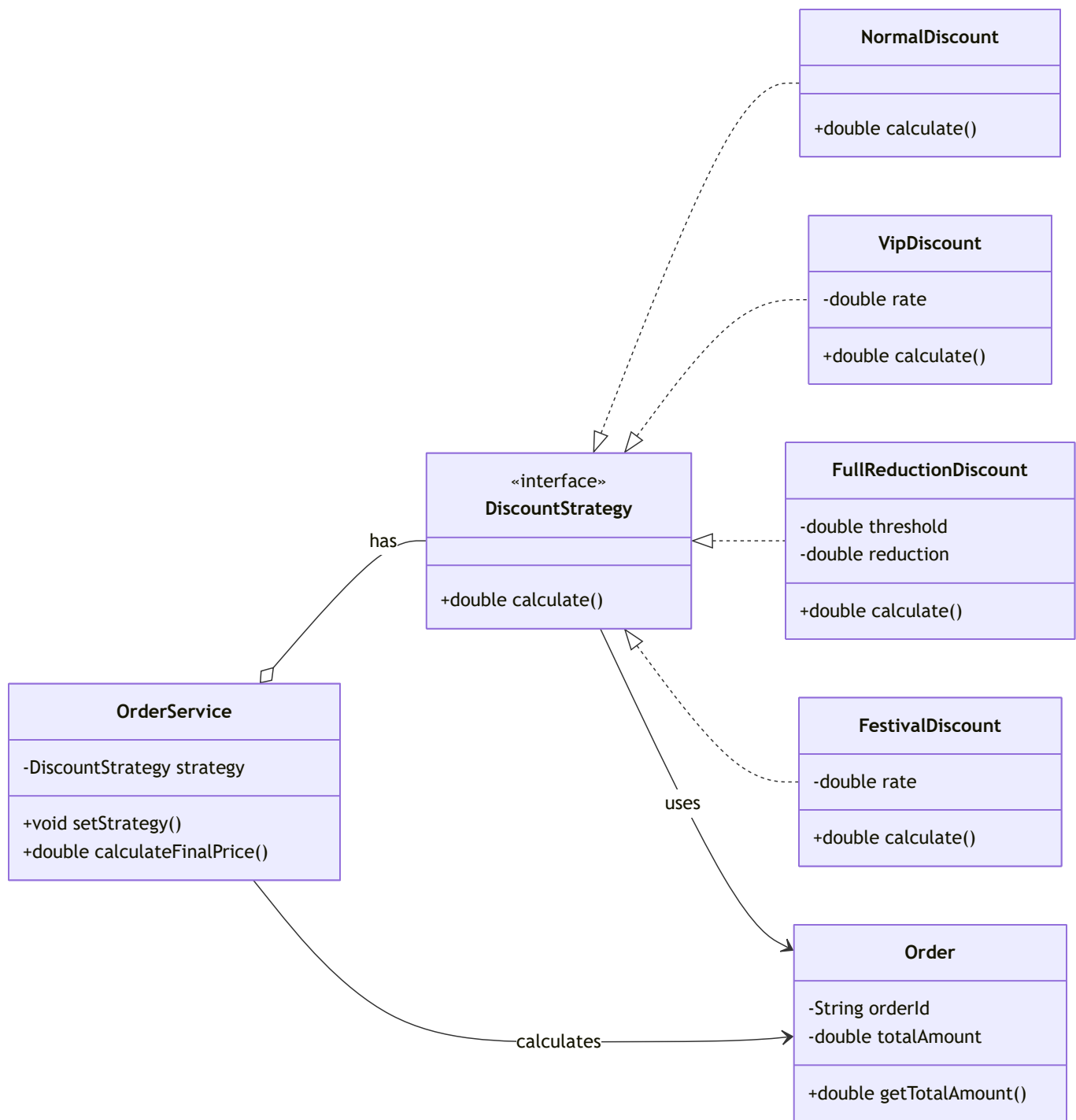
小题背景

一个电商系统中，订单可以使用不同折扣规则，例如普通价格、VIP 折扣、满减折扣、节日折扣。系统不希望 在 `OrderService` 中写大量 `if / else` 判断折扣类型。

应使用的模式

Strategy 策略模式

Mermaid UML



设计说明

不同折扣规则本质上是一组可替换算法。将每种折扣封装成一个具体策略类，`OrderService` 只依赖 `DiscountStrategy` 接口。新增折扣时只新增策略类，不修改 `OrderService`，避免大量条件判断，符合开闭原则。

4. Iterator 迭代器模式：播放列表遍历

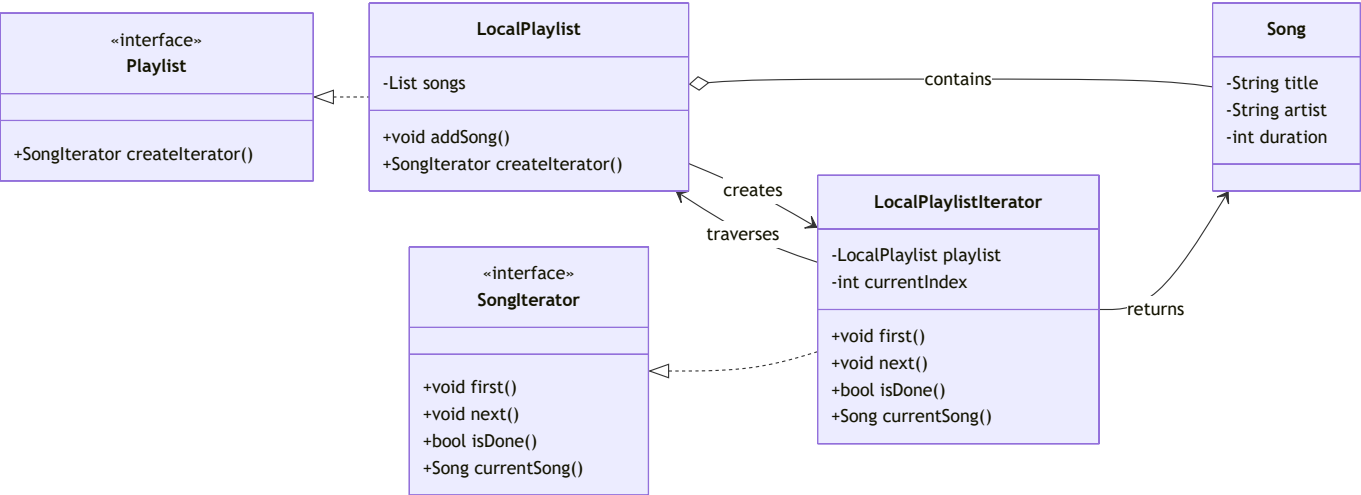
小题背景

一个音乐播放器中，播放列表内部可能用数组、链表或数据库结果保存歌曲。客户端只想按顺序遍历歌曲，而不想知道播放列表的内部存储方式。

应使用的模式

Iterator 迭代器模式

Mermaid UML



设计说明

播放列表负责保存歌曲，迭代器负责遍历歌曲。客户端通过 `SongIterator` 的统一接口访问歌曲，不需要知道 `LocalPlaylist` 内部如何存储数据。这样遍历逻辑和集合存储逻辑分离，后续更换存储结构时不会影响客户端。

5. Adapter 适配器模式：第三方支付接口适配

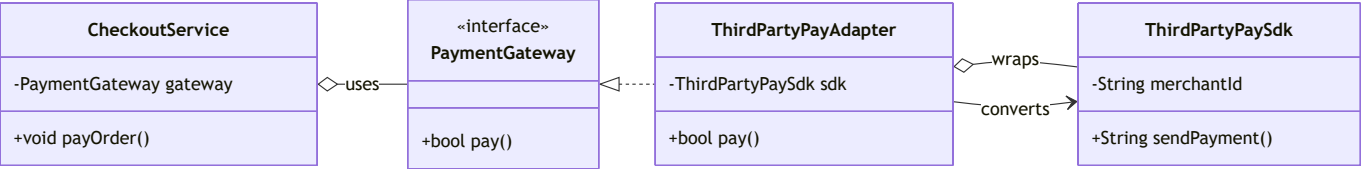
小题背景

系统原本只支持统一的 `PaymentGateway` 接口，但现在要接入一个第三方支付 SDK。第三方 SDK 的方法名和参数格式与系统接口不一致，且不能修改第三方 SDK 源码。

应使用的模式

Adapter 适配器模式

Mermaid UML



设计说明

客户端 `CheckoutService` 只认识系统定义的 `PaymentGateway`。由于第三方 SDK 接口不兼容，因此增加 `ThirdPartyPayAdapter`，在适配器中把 `pay()` 转换为第三方 SDK 所需的 `sendPayment()`。这样既复用了旧接口，又不需要修改第三方类。

6. Composite 组合模式：文件系统目录树

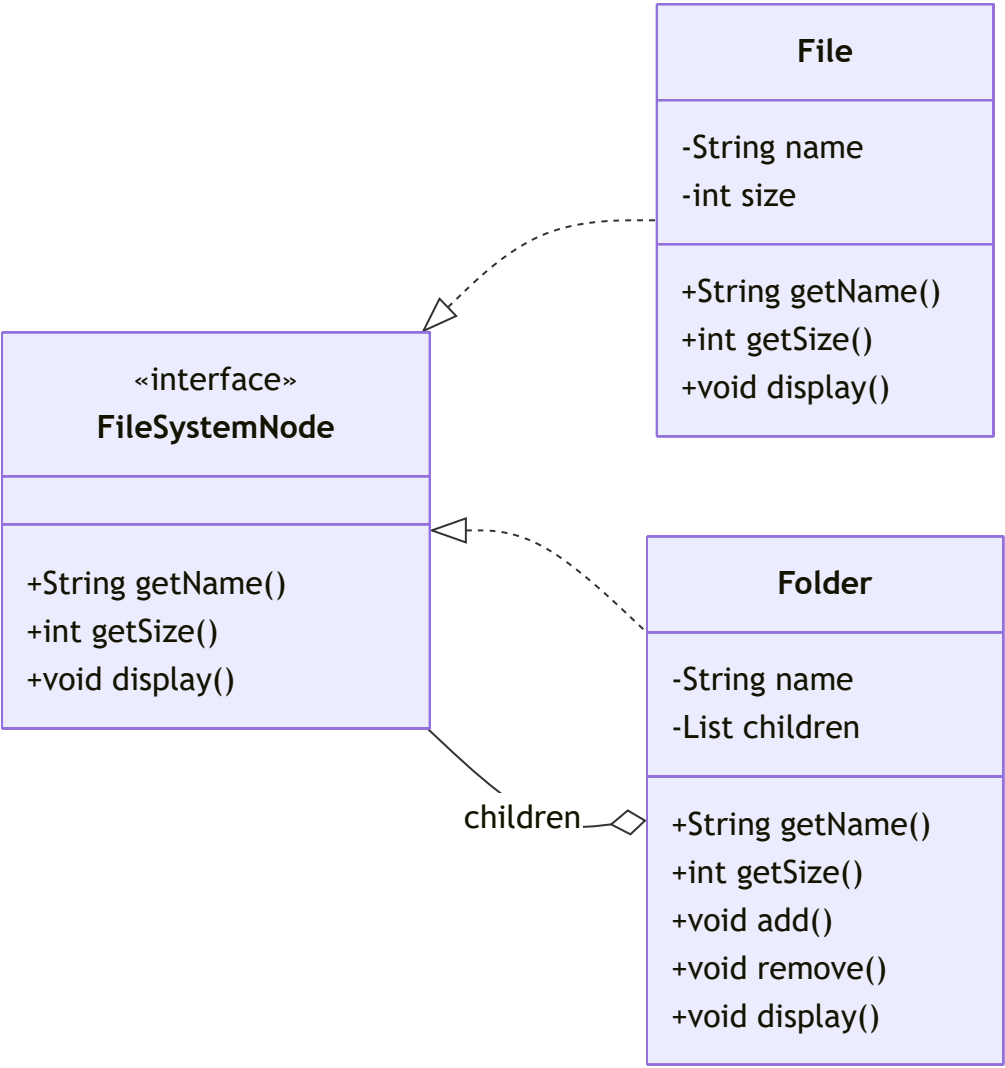
小题背景

一个文件管理器中既有普通文件，也有文件夹。文件夹可以包含文件和子文件夹。客户端希望统一计算文件或文件夹的大小，而不区分它是单个文件还是目录。

应使用的模式

Composite 组合模式

Mermaid UML



设计说明

普通文件是叶子节点，文件夹是组合节点。二者都实现 `FileSystemNode` 接口，因此客户端可以统一调用 `getSize()` 和 `display()`。 `Folder` 的大小由其所有子节点大小递归求和。该模式适合树形结构，并能统一处理“部分”和“整体”。

7. Command 命令模式：文本编辑器撤销操作

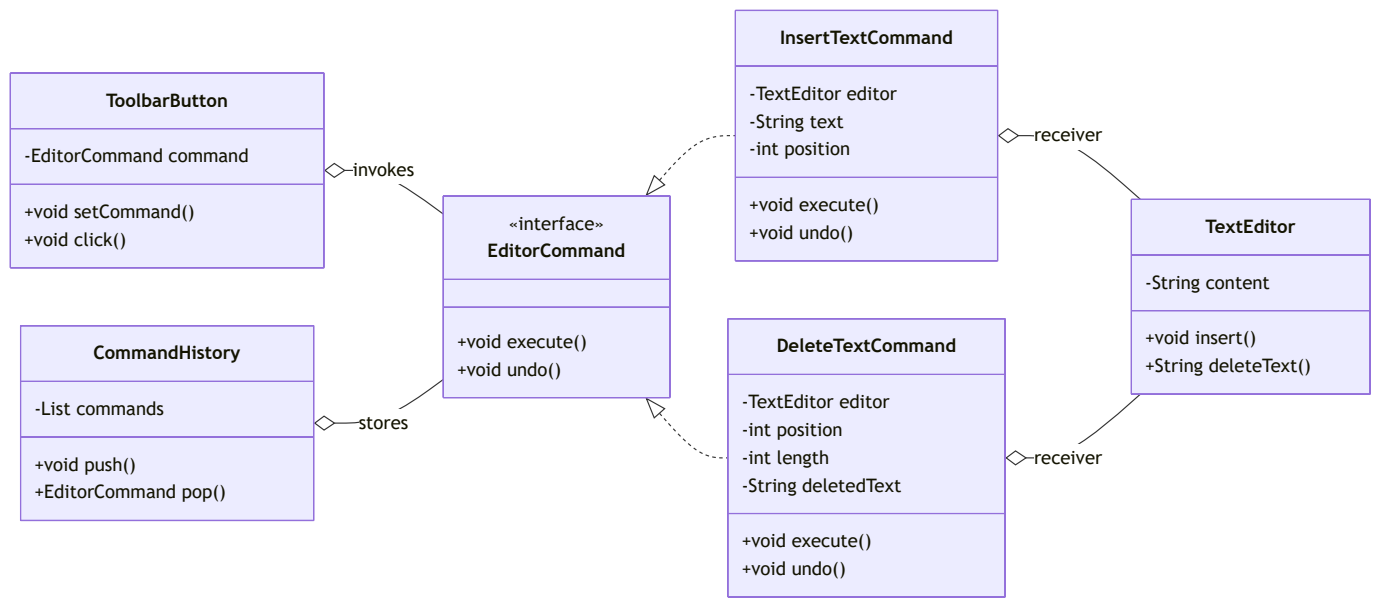
小题背景

一个文本编辑器支持插入文字、删除文字、复制粘贴等操作，并希望支持撤销、重做和操作历史记录。系统不希望按钮直接调用编辑器对象的具体方法。

应使用的模式

Command 命令模式

Mermaid UML



设计说明

命令模式把一次编辑操作封装成一个命令对象。按钮只调用 `execute()`，不直接知道具体编辑逻辑。命令对象内部保存接收者 `TextEditor` 以及执行所需参数，因此可以被保存到历史记录中，用于撤销和重做。该模式适合按钮操作、菜单命令、事务、撤销/重做等场景。

8. Decorator 装饰器模式：咖啡加料计价

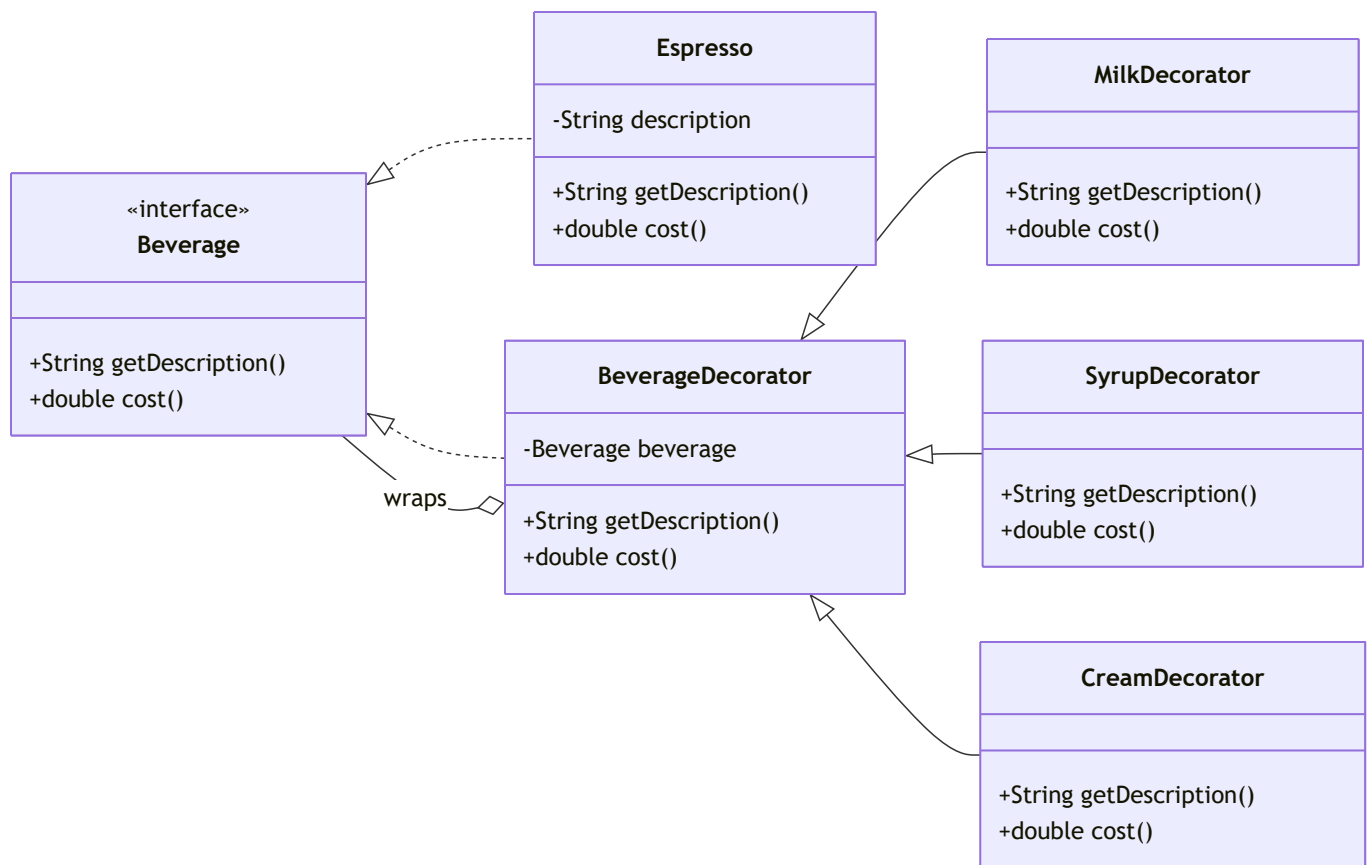
小题背景

一家咖啡店售卖基础咖啡，并允许顾客动态添加牛奶、糖浆、奶油等配料。系统不希望为每一种组合都创建一个子类，例如“牛奶咖啡”“牛奶糖浆咖啡”“牛奶糖浆奶油咖啡”。

应使用的模式

Decorator 装饰器模式

Mermaid UML



设计说明

基础咖啡和所有装饰器都实现 `Beverage` 接口。装饰器内部持有一个 `Beverage` 对象，并在原有价格和描述基础上追加自己的功能。这样可以运行时动态组合配料，例如先包一层牛奶，再包一层糖浆，再包一层奶油。该模式比继承更灵活，适合动态增强对象功能。

9. Singleton 单例模式：系统日志管理器

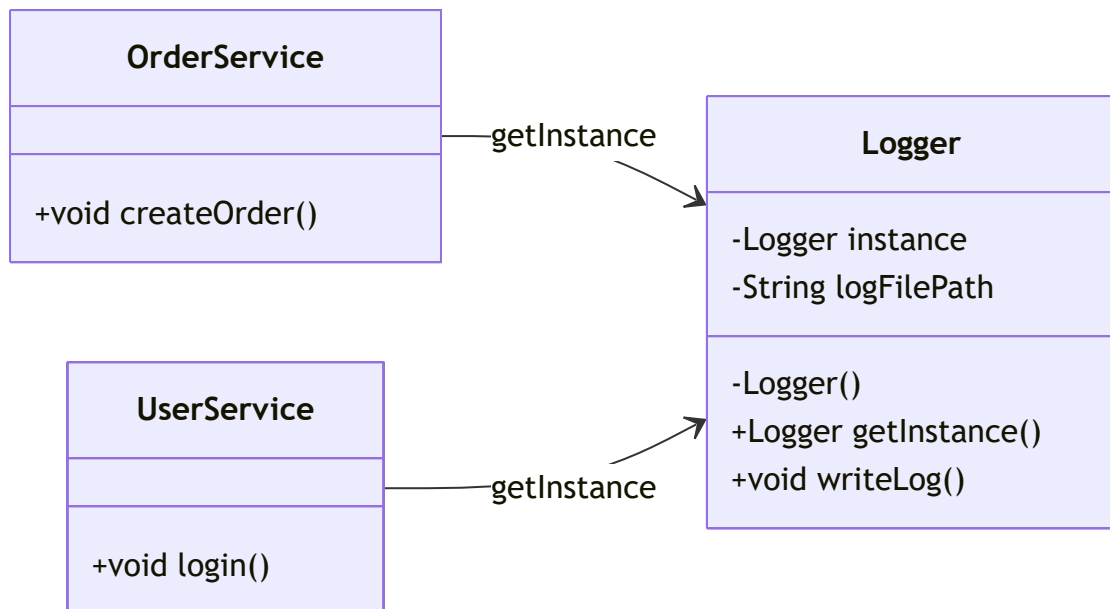
小题背景

一个后台系统需要统一记录运行日志。为了避免多个日志对象同时写同一个日志文件导致混乱，系统要求全局只能有一个 `Logger` 实例。

应使用的模式

Singleton 单例模式

Mermaid UML



设计说明

`Logger` 构造函数私有化，外部不能直接创建对象，只能通过 `getInstance()` 获取唯一实例。这样系统中所有模块都使用同一个日志对象。单例适合配置管理器、日志管理器、线程池等需要唯一共享资源的场景。但它可能造成全局状态和隐藏依赖，不利于测试，所以不能滥用。

10. Abstract Factory 抽象工厂模式：跨平台 UI 组件

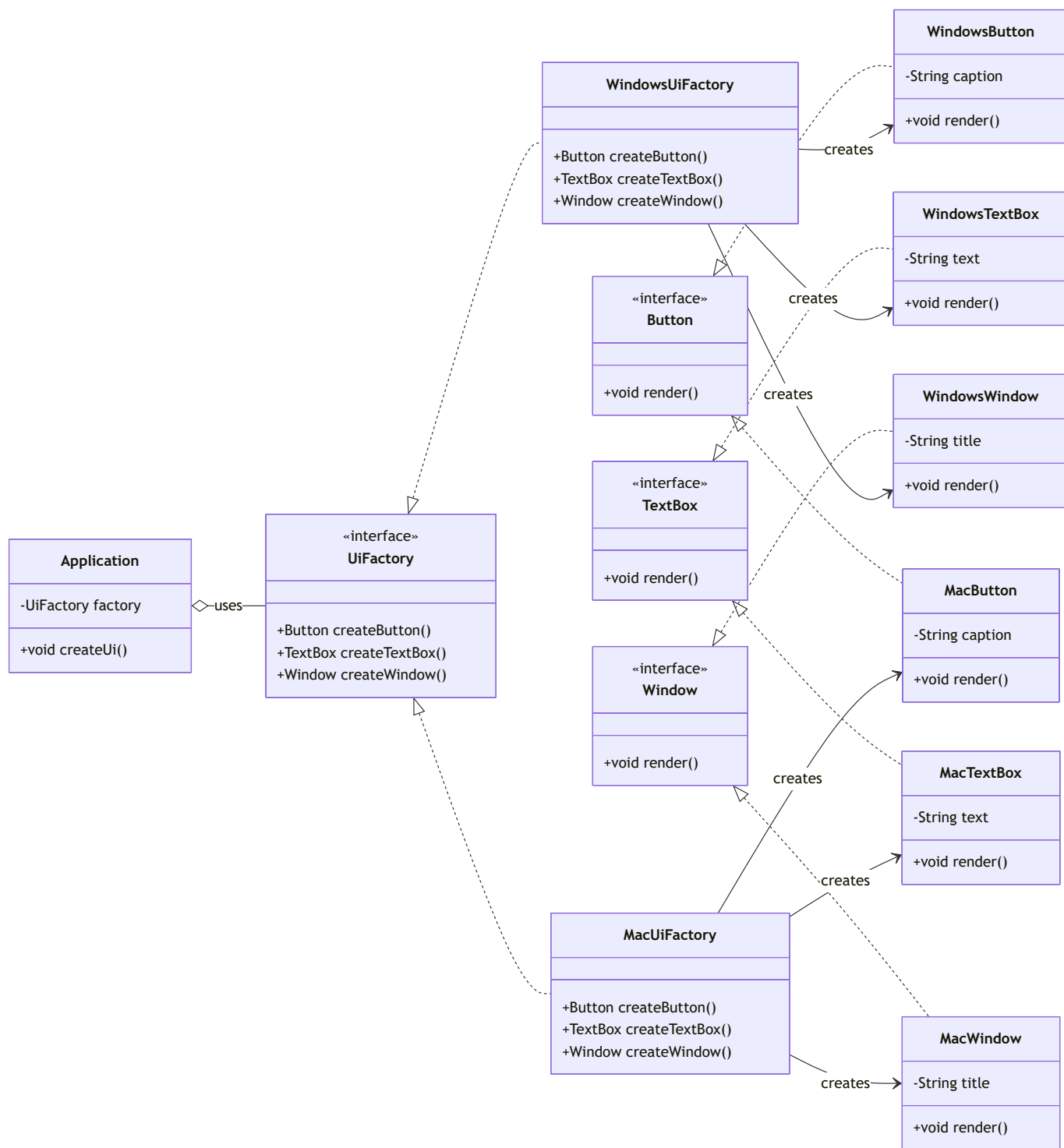
小题背景

一个桌面应用需要支持 Windows 和 macOS 两套界面风格。每套风格都包含按钮、文本框和窗口。系统希望切换平台风格时，所有 UI 组件都属于同一产品族。

应使用的模式

Abstract Factory 抽象工厂模式

Mermaid UML



设计说明

抽象工厂用于创建一族相关对象。`Application` 只依赖抽象工厂 `UiFactory` 和抽象产品 `Button`、`TextBox`、`Window`。如果使用 `WindowsUiFactory`，就创建 Windows 风格组件；如果使用 `MacUiFactory`，就创建 Mac 风格组件。它可以保证同一产品族的一致性，也方便切换整套产品族。缺点是新增产品种类时需要修改所有工厂。

附录：DI 依赖注入：订单服务发送通知

小题背景

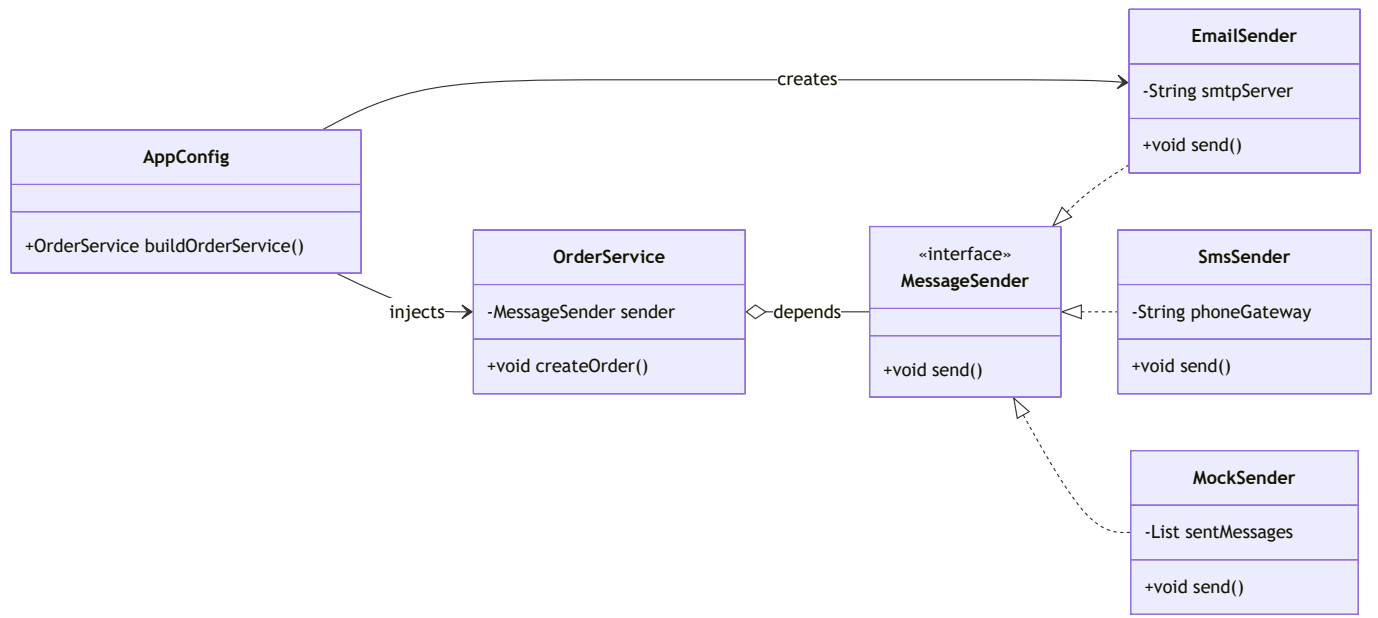
一个订单系统在创建订单后需要发送通知。通知方式可能是邮件、短信或测试用的 Mock 通知。系统不希望 `OrderService` 在内部直接创建具体通知对象。

相关设计思想

DI 依赖注入

它不是 GoF 设计模式，但常和 DIP、TDD、Mock Object、可测试性一起考。

Mermaid UML



设计说明

`OrderService` 不在内部直接创建 `EmailSender`，而是通过构造函数接收一个 `MessageSender`。这样业务类只依赖抽象接口，不依赖具体实现，符合依赖倒置原则。运行时可以注入 `EmailSender` 或 `SmsSender`，测试时可以注入 `MockSender`。这样系统更容易扩展，也更方便单元测试。

快速识别表

小题背景关键词	优先考虑模式
AST 节点稳定，但要增加多种分析操作	Visitor
股票价格变化后通知多个显示器	Observer
多种折扣规则，避免 if / else	Strategy
播放列表遍历，不暴露内部结构	Iterator
第三方 SDK 接口不兼容	Adapter
文件和文件夹形成树形结构	Composite
编辑器操作要支持撤销、重做、历史记录	Command

小题背景关键词	优先考虑模式
咖啡动态添加配料，不想类爆炸	Decorator
全局只有一个日志对象	Singleton
创建一整套 Windows / Mac UI 组件	Abstract Factory
外部注入邮件、短信、Mock 通知对象	DI